

第二章 - 变量、数据类型、表达式 (编程练习)

张建章

阿里巴巴商学院

杭州师范大学

2026-09



1 变量与赋值

2 基本数据类型

3 表达式

4 输入、计算与输出

5 商业计算与测试

6 课后拓展



用 Python
让商业想法
变成现实!

Python 程序设计基础

第2章 变量、数据类型、表达式 速记卡

用简单的代码，解决身边的商业问题!

学会基础
从小例子开始，
你也可以!

数据 + 思考
= 更好的
商业决策!

1 这一章学什么?

- ✓ 学会用变量保存数据
- ✓ 认识 int、float、str、bool、None
- ✓ 会写简单的表达式
- ✓ 掌握类型转换 (数据类型之间的转换)
- ✓ 学会输入和输出
- ✓ 会用简单的测试检查结果是否正确

把业务数据变成代码，
先算对，再做大!

2 变量与赋值

- 变量 = 给数据起名字
- 左边是变量名，中间是 =，右边是数据或表达式
- Python 先计算右边的表达式，再把结果赋值给左边的变量

```
order_id = "CN-2026-0908"  
unit_price_usd = 29.90  
quantity = 3  
subtotal_usd = unit_price_usd * quantity
```

变量的值会随着程序运行而改变。

3 变量命名小贴士

- ✓ 用小写英文字母
- ✓ 多个单词用下划线连接
- ✓ 起有意义的业务相关名字
- ✓ 不能以数字开头
- ✓ 不能有空格或连字符 (-)
- ✓ 避免使用 Python 关键字或内置函数名

✓ 好的例子

```
unit_price_usd  
exchange_rate  
order_total  
is_paid
```

✗ 不好的例子

```
2nd_order  
unit-price  
order total  
class
```

4 5种基础数据类型

int	整数，如 quantity = 3
float	小数，如 unit_price_usd = 29.90
str	字符串，如 order_id = "CN-2026-0908"
bool	布尔值，如 is_paid = True
None	空值，如 refund_reason = None

不同类型的数据，用在不同的场景中，选择合适的类型很重要!

5 表达式与类型转换

A. 常见的表达式

- 算术运算: + - * / ** %
- 比较运算: == != > < >= <=
- 逻辑运算: and or not

B. 类型转换函数

- int() 将字符串转为整数
- float() 将字符串转为小数
- str() 将其他类型转为字符串

input() 得到的是字符串，
在计算前需要先进行类型转换!

6 输入、计算与输出

输入 → 类型转换 → 计算 → 输出

```
input() int() float() str() 表达式 print()
```

```
# 1. 输入  
unit_price_usd = float(input("请输入单价(美元): "))  
quantity = int(input("请输入购买数量: "))  
# 2. 计算  
subtotal = unit_price_usd * quantity  
# 3. 输出 (保留两位小数)  
print(f"小计: {subtotal:.2f} 美元")
```

7 商业计算小例子

计算订单应付金额 (美元 → 人民币)

已知数据:

- 单价: 29.90 美元
- 数量: 3
- 优惠: 5.00 美元
- 汇率: 7.12

计算步骤:

- 1 小计: 29.90 * 3 = 89.70
- 2 扣减优惠: 89.70 - 5.00 = 84.70
- 3 换算人民币: 84.70 * 7.12
- 4 结果: 603.06 元

代码实现:

```
price = 29.90  
num = 3  
discount = 5.00  
rate = 7.12  
amount_cny = (price * num - discount) * rate  
print(f"应付金额: {amount_cny:.2f} 元")
```

结果: 603.06 元

8 测试与学习提醒

- ✓ 用不同的数据测试程序;
- ✓ 正常计算、无优惠、数量为 0
- ✓ 测试小数量数据，如 19.9
- ✓ 测试负数数量 (看看是否需要限制)
- ✓ 测试优惠金额大于总金额 (思考如何处理)
- ✓ 注意短路逻辑 (and / or 和括号的使用)
- ✓ 可以用 AI 作为学习助手，但自己要验证业务规则和计算结果是否正确

小贴士:

多做测试，敢于试错，
才能真正掌握 Python!

用变量记录一笔跨境订单

订单中的业务数据

- 订单编号: CN-2026-0908
- 商品名称: 蓝牙耳机
- 商品单价: 29.90美元
- 购买数量: 3件
- 优惠金额: 5.00美元
- 管理汇率: 7.12

在Python程序中记录数据

```
order_id = "CN-2026-0908"  
product_name = "蓝牙耳机"  
  
unit_price_usd = 29.90  
quantity = 3  
discount_usd = 5.00  
exchange_rate = 7.12
```

程序首先用**有意义的名称**记录完成任务所需的数据。

观察：每一行代码可以分成哪三个部分？

变量与赋值

变量是程序中用于引用数据的名称。初学时可以把变量理解为 **给一项数据起名字**，方便程序重复使用这项数据。

```
unit_price_usd = 29.90
quantity = 3

subtotal_usd = unit_price_usd * quantity
```

赋值语句的三个部分

- 左侧：变量名
- 中间：赋值符号 `=`
- 右侧：数据或计算表达式

Python的执行过程

- ① 读取右侧变量的当前值
- ② 计算右侧表达式
- ③ 将结果赋给左侧变量

赋值语句的阅读方法

`=` 表示赋值。Python先计算右侧，再把结果交给左侧变量。

变量的值会随程序运行而改变

某商品当前有50件库存，随后连续售出3件和5件：

```
stock = 50
stock = stock - 3
stock = stock - 5

stock
```

次序	执行的语句	stock 当前值
1	stock = 50	50
2	stock = stock - 3	47
3	stock = stock - 5	42

执行 `stock = stock - 3` 时，Python先读取 `stock` 原来的值50，计算 `50 - 3`，再将结果47重新赋给 `stock`。

Notebook中的变量状态

变量的当前值由代码的**实际运行顺序**决定。如果运行结果与预期不同，可以重新启动内核，再从上到下运行全部单元格（点那个快进按钮/run all cells）。

变量命名

Python的语法要求

- 可以使用字母、数字和下划线
- 不能以数字开头
- 不能包含空格、连字符等符号
- 不能使用Python关键字
- 区分大小写，例如price和Price是两个变量

本课程的命名习惯

- 使用小写英文单词
- 多个单词之间使用下划线
- 名称应准确表达业务含义
- 不能使用Python保留关键字，例如，for、if、return等
- 避免覆盖Python内置名称，例如，str、float、sum。

`unit_price_usd``exchange_rate``amount_cny`

好的变量名使代码接近一份可以阅读的业务计算说明。

变量命名练习

下面左侧的变量名存在哪些问题？右侧是一种参考修改。

需要修改

```
2nd_order_id = "A002"
```

```
unit-price = 29.90
```

```
exchange rate = 7.12
```

```
float = 5.00
```

```
a = 84.70
```

一种参考修改

```
second_order_id = "A002"
```

```
unit_price_usd = 29.90
```

```
exchange_rate = 7.12
```

```
discount_usd = 5.00
```

```
amount_usd = 84.70
```

- 前三个变量名违反Python的命名语法，程序无法运行
- `float` 和 `a` 语法有效，但无法清楚表达业务含义

高质量变量名同时满足：语法有效，业务含义清楚。

一笔订单包含不同类型的数据

数据类型说明Python应当如何理解一项数据，以及可以对它进行哪些操作。

```
order_id = "CN-2026-0908"  
quantity = 3  
unit_price_usd = 29.90  
is_paid = True  
refund_reason = None
```

- `int`：整数
- `float`：浮点数
- `str`：字符串
- `bool`：布尔值
- `None`：表示没有值

选择数据类型的依据

一项数据的业务含义和后续操作共同决定它的类型。订单编号虽然包含数字，但它用于标识订单，因此应当保存为字符串。

严格地说，`None` 是Python中的一个特殊值，它的类型是 `NoneType`。

整数与浮点数

整数 `int`

```
quantity = 3  
stock = 50  
order_count = 128
```

```
type(quantity) # int
```

常见业务数据包括：

- 商品数量
- 库存数量
- 订单数量

浮点数 `float`

```
unit_price_usd = 29.90  
exchange_rate = 7.12  
discount_rate = 0.15
```

```
type(unit_price_usd) # float
```

常见业务数据包括：

- 商品价格和汇率
- 折扣率和增长率
- 平均值

浮点数的近似表示（浮点数的不定尾数现象）

计算机使用二进制近似保存浮点数，许多十进制小数无法被精确表示。例如， $0.1 + 0.2$ 的结果可能显示为 `0.30000000000000004`。自己算算 7.5×5.31 。

字符串

字符串 `str` 用于表示文本。字符串内容需要放在引号中。

```
order_id = "CN-2026-0908"  
product_name = "蓝牙耳机"  
country_code = "CN"  
product_code = "000872"  
  
type(order_id) # str
```

订单编号、商品编码等即使全部由数字组成，通常也应当保存为字符串，因为它们用于标识，不用于算术计算。

```
quantity = 3  
quantity_text = "3"
```

3 表示数量，"3" 表示文本字符。

引号属于Python语法

变量名不加引号，字符串内容都需要加引号。单引号、双引号、三引号都可以表示字符串，但两端必须配对。

思考：如果字符串内容本身包含引号，如何处理？

布尔值与None

布尔值 bool

布尔值用于表示只有两种可能结果的状态，取值为 `True` 或 `False`。

```
is_paid = True
is_refunded = False
has_coupon = True

type(is_paid) # bool
```

变量名通常写成可以回答“是或否”的问题，例如“是否付款”。

特殊值 None

`None` 表示当前没有可用的值，可以表示尚未填写、尚未发生或不适用。

```
payment_time = None
refund_reason = None

type(refund_reason) # NoneType
```

`None` 与字符串 `"None"` 含义不同。后者只是由四个字母组成的文本。

书写规则

`True`、`False` 和 `None` 的**首字母必须大写**，并且不需要添加引号。

用type()检查数据类型

内置函数 `type()` 可以查看一项数据的类型。

```
product_code = "000872"  
quantity = 3  
discount_rate = 0.10  
is_member = False  
coupon_code = None
```

```
type(product_code) # str  
type(quantity) # int  
# float  
type(discount_rate)  
type(is_member) # bool  
# NoneType  
type(coupon_code)
```

选择类型的基本思路

需要算术计算的数据使用 `int` 或 `float`；文本和编码使用 `str`；两种状态使用 `bool`；暂时没有值时可以使用 `None`。

类型转换

类型转换使用内置函数把一个值转换为另一种类型。

```
quantity_text = "3"  
quantity = int(quantity_text)  
  
unit_price_text = "29.90"  
unit_price_usd =  
    ↪ float(unit_price_text)  
  
order_number = 20260908  
order_id = str(order_number)
```

- `int()` 转换为整数
- `float()` 转换为浮点数
- `str()` 转换为字符串

类型转换会**产生一个新值**。原来的变量仍然保留原来的值和类型。

```
type(quantity_text) # str  
type(quantity) # int
```

字符串必须符合目标类型的格式

`int("3")` 和 `float("29.90")` 可以正常转换。

`int("three")` 和 `float("29.90美元")` 会产生错误。

从网页表单、文件或其他系统获得的数据经常是字符串。参与计算之前，需要先把文本转换为合适的数值类型。

```
# 原始数据都是字符串
unit_price_text = "29.90"
quantity_text = "3"
discount_text = "5.00"
# 转换为可以计算的数值
unit_price_usd = float(unit_price_text)
quantity = int(quantity_text)
discount_usd = float(discount_text)
# 按照业务规则计算
subtotal_usd = unit_price_usd *
    ↪ quantity
amount_usd = round(
    subtotal_usd - discount_usd, 2
)
amount_usd # 84.7
```

阅读代码时依次检查

- ❶ 原始数据是什么类型？
- ❷ 每项数据需要转换为什么类型？
- ❸ 表达式是否符合业务规则？
- ❹ 计算结果是否合理？

尝试回答

- 如果不进行类型转换，会发生什么？
- 数量可以使用 `float` 吗？
- 优惠金额大于商品总价时，结果合理吗？

上例总结：先确认数据含义和类型，再按照业务规则编写（算数）表达式。

算数表达式

表达式由数据和操作符组成。Python计算表达式后会得到一个值。
算数表达式由数据和算数操作符组成。

表 1: 算数操作符

操作符	含义及示例
+	加法: 收入加运费
-	减法: 总价减优惠
*	乘法: 单价乘数量
/	除法: 销售额除以订单数
//	整除: 商品可以装满几箱
%	取余: 装箱后还剩几件
**	幂运算: 计算复合增长

```
unit_price_usd = 29.90
quantity = 3
discount_usd = 5.00

subtotal_usd = unit_price_usd
↳ * quantity

amount_usd = subtotal_usd -
↳ discount_usd
```

- `unit_price_usd * quantity` 是表达式, 表达式计算后得到商品总价
- 赋值语句 `subtotal_usd = unit_price_usd * quantity` 把结果交给变量

比较表达式与逻辑表达式

比较表达式

比较两个值，计算结果为True或False。

操作符	含义
<code>==</code>	等于
<code>!=</code>	不等于
<code>></code>	大于
<code><</code>	小于
<code>>=</code>	大于或等于
<code><=</code>	小于或等于

```
amount_usd = 84.70
quantity = 3
amount_usd >= 100  # False
quantity == 3      # True
```

逻辑表达式

逻辑操作符可以组合多个条件。

- `and`：两个条件都成立
- `or`：至少一个条件成立
- `not`：反转原来的结果

```
amount_usd = 120
quantity = 4
is_paid = True
is_refunded = False
is_large_order = (
    amount_usd >= 100
    and quantity >= 3
)
can_ship = is_paid and not
    ↪ is_refunded
```

`=` 把右侧结果赋给左侧变量，`==` 比较左右两侧的值是否相等。

使用input()接收用户输入

内置函数 `input()` 用于接收键盘输入。运行到 `input()` 时，程序会暂停并等待用户输入，按下回车键后继续执行。

假设用户输入：

```
quantity_text = input(  
    "请输入购买数量："  
)
```

```
quantity_text  
type(quantity_text)
```

3

变量的值和类型分别是：

```
'3'  
str
```

`input()`的返回值

`input()` 接收的结果始终是字符串。用户输入 3 ,Python得到的是字符串 "3" 。

把输入转换为可以计算的数据

字符串形式的数字需要转换为 `int` 或 `float`，才能按照数值参与计算。

分步写法

```
quantity_text =
↪ input("请输入购买数量: ")
quantity = int(quantity_text)

unit_price_text =
↪ input("请输入商品单价: ")
unit_price_usd =
↪ float(unit_price_text)
subtotal_usd = (unit_price_usd *
↪ quantity)
```

紧凑写法

```
quantity = int(input("请输入
↪ 入购买数量: "))

unit_price_usd = float(inp
↪ ut("请输入商品单价: "))

subtotal_usd =
↪ (unit_price_usd *
↪ quantity)
```

初学时，分步写法更容易观察变量的值和类型，也更方便定位错误。

使用print()和f-string输出结果

`print()` 用于输出程序结果。f-string可以把变量值放入一段说明文字中。

运行结果

人民币实付金额: 603.064

订单CN-2026-0908的人民
币实付金额为603.06元

```
order_id = "CN-2026-0908"
amount_cny = 603.064

print("人民币实付金额: ", amount_cny)

print(
    f"订单{order_id}的人民币实付金额"
    f"为{amount_cny:.2f}元"
)
```

f-string的写法

- 字符串前添加 `f`
- 花括号中填写变量
- `:.2f` 显示两位小数

显示格式与变量值

`:.2f` 只控制输出时的显示格式。变量 `amount_cny` 保存的值仍然是 `603.064` 。

完整的订单金额计算程序

```
unit_price_usd =  
    ↪ float(input("请输入商品单价（美元）："))  
quantity = int(input("请输入购买数量："))  
discount_usd = float(input("请输入优惠金额  
    ↪ （美元）："))  
exchange_rate =  
    ↪ float(input("请输入管理汇率："))  
  
subtotal_usd = unit_price_usd * quantity  
amount_usd = subtotal_usd - discount_usd  
amount_cny = round(amount_usd *  
    ↪ exchange_rate, 2)  
  
print(f"人民币实付金额：{amount_cny:.2f}元")
```

一次运行

商品单价：29.90

购买数量：3

优惠金额：5.00

管理汇率：7.12

输出结果：

人民币实付金额：603.06元

运行前先检查

- 输入数据是否完整？
- 类型转换是否正确？
- 计算公式是否符合规则？
- 输出是否带有单位？

商业计算需要明确的验收标准

业务任务

根据商品单价、购买数量、优惠金额和管理汇率，计算订单的人民币实付金额：

$$\text{人民币实付金额} = (\text{美元单价} \times \text{数量} - \text{美元优惠}) \times \text{管理汇率}$$

已知数据

- 商品单价：29.90 美元；
- 购买数量：3 件；
- 优惠金额：5.00 美元；
- 管理汇率：7.12 。

验收标准

- 每项输入的含义和单位明确；
- 计算过程符合业务规则；
- 最终结果保留两位小数；
- 输出包含币种和金额单位。

基准结果

这组已知数据的正确结果应为：人民币实付金额 603.06 元。它将作为检查程序是否正确的第一组基准数据。

使用已知结果进行基准测试

```

unit_price_usd = 29.90
quantity = 3
discount_usd = 5.00
exchange_rate = 7.12

subtotal_usd = unit_price_usd *
↳ quantity
amount_usd = subtotal_usd -
↳ discount_usd
actual_cny = round(amount_usd *
↳ exchange_rate, 2)

expected_cny = 603.06

print(f"实际结果: {actual_cny:.2f}元")
print(f"测试通过: {actual_cny ==
↳ expected_cny}")

```

人工计算

$$29.90 \times 3 = 89.70$$

$$89.70 - 5.00 = 84.70$$

$$84.70 \times 7.12 = 603.064$$

$$\text{保留两位小数} = 603.06$$

实际结果: 603.06元

测试通过: True

测试的基本思想

先确定预期结果，再运行程序得到实际结果，最后比较二者是否一致。一组数据通过，只能证明程序正确处理了这一种情况。

测试需要覆盖不同的业务场景

以下各组数据使用相同的计算公式，金额输入单位均为美元：

场景	单价	数量	优惠	汇率	业务预期
正常订单	29.90	3	5.00	7.12	603.06
无优惠	29.90	3	0.00	7.12	638.66
数量为0	29.90	0	0.00	7.12	0.00
小数金额	0.10	3	0.00	7.12	2.14
负数数量	29.90	-2	0.00	7.12	应拒绝
优惠超总价	29.90	3	100.00	7.12	应拒绝

不同测试数据承担不同任务

正常数据检查基本计算，零值和小数检查边界情况，负数数量和过高优惠检查业务规则。这些数据共同回答一个问题：**程序在不同情况下是否仍然可信？**

当前程序遇到最后两组数据时仍会继续计算，并得到负数金额。这说明**算术公式虽然正确，程序却尚未完整表达业务规则**。本节先通过测试发现问题，后续再实现输入校验。

使用AI智能体扩展测试场景

可直接交给AI智能体的Prompt

我有一个Python程序，按照下面的公式计算人民币实付金额：

$$(\text{美元单价} \times \text{数量} - \text{美元优惠}) \times \text{管理汇率}$$

请只设计6组测试数据，不修改程序。

测试应覆盖正常订单、无优惠、数量为0、负数数量、优惠超过商品总价和小数金额。

每组列出4项输入、预期结果和测试理由，并完整展示其中1组的计算过程。

需要完成的人工核验

- 输入项目和单位是否完整；
- 人工复算至少两组结果；
- 使用程序逐组运行测试；
- 记录预期结果和实际结果；
- 判断结果是否符合业务常识；
- 找出程序尚未表达的规则。

人与AI智能体的分工

AI智能体可以快速扩展测试思路；人需要核对业务规则、数据单位和预期结果，并对测试结论负责。

运算符优先级：理解常用规则，不必背表

本章常用运算符的优先级

从高到低	运算符
1	()
2	**
3	+x 、 -x
4	* 、 / 、 // 、 %
5	+ 、 -
6	比较运算符
7	not
8	and
9	or

最实用的规则

- 先计算括号中的内容；
- 乘除通常先于加减；
- 比较运算先于逻辑运算；
- not 、 and 、 or 依次计算；
- 不确定时主动添加括号。

一个容易出错的例子

-2 ** 2 的结果为 -4 ；

(-2) ** 2 的结果为 4 。

学习建议

不需要背诵完整优先级表。遇到复杂表达式时，使用括号明确计算顺序，或者把表达式拆成多个步骤。

短路运算：为什么后半段没有执行

业务问题：计算平均成交价之前，需要确认销售数量不为0。下面的表达式用于判断平均成交价是否达到100元：

```
sales = 3000
quantity = 0
quantity != 0 and sales / quantity >= 100
```

Python的执行过程

- 先计算 `quantity != 0`；
- 因为 `quantity` 为0，结果是 `False`；
- 整个 `and` 表达式已经确定为 `False`；
- Python不再计算 `sales / quantity`；
- 因此不会发生除以0的错误。

and的短路规则

对于 `A and B`，如果 `A` 为假，Python不再计算 `B`。

or的短路规则

对于 `A or B`，如果 `A` 为真，Python不再计算 `B`。

短路运算的作用：短路运算可以减少不必要的计算，也可以把必要的检查放在前面，避免执行可能产生错误的表达式。

复杂逻辑式：拆开写，更容易核验

一条不容易核验的表达式

```
can_discount = price > 0 and quantity > 0 and amount >= 500 or is_member
```

隐藏的业务风险

由于 `and` 的优先级高于 `or` ,只要 `is_member` 为 `True` ,整个表达式就可能得到 `True` ,即使商品价格或数量不合理。这可能并不符合“数据有效时才能享受优惠”的业务规则。

把业务规则拆成可执行步骤

```
data_valid = price > 0 and quantity > 0  
discount_allowed = amount >= 500 or is_member  
can_discount = data_valid and discount_allowed
```

拆分后的好处：把复杂业务逻辑拆成有意义的判断，分别命名后再组合结果，使代码更容易理解、测试。

THE END